

Minimizing order picking makespan with multiple pickers in a wave picking warehouse



Ehsan Ardjmand^{a,*}, Heman Shakeri^b, Manjeet Singh^c, Omid Sanei Bajgiran^d

^a Management Department, College of Business, Frostburg State University, Frostburg, MD, USA

^b Electrical and Computer Engineering Department, Kansas State University, Manhattan, KS, USA

^c Solutions Design, DHL Supply Chain, Westerville, OH, USA

^d Rotman School of Management, University of Toronto, Toronto, Canada

ARTICLE INFO

Keywords:

Order picking
Order batching
Routing optimization
Order assignment
Lagrangian decomposition
Metaheuristics

ABSTRACT

In this paper, order assignment, order batching and picker routing problems with multiple pickers in a wave picking warehouse of a major US third party logistics company is studied and modeled mathematically. The proposed mathematical model is solved using an exact algorithm. Since the exact algorithm suffers from long CPU time, a Lagrangian decomposition heuristic combined with particle swarm optimization (LD-PSO) algorithm is proposed, which performs well for small size waves. To solve large-scale problems, a hybrid parallel simulated annealing and ant colony optimization (PSA-ACO) is presented. The proposed methods are tested using the warehouse data. The results are compared against the minimum makespan impact (MMI) heuristic that is currently being used in the warehouse and a state-of-the-art variable neighborhood descent (VND). While PSA-ACO slightly outperforms VND, for picking large waves, PSA-ACO and VND can improve the makespan by approximately 7.8% and 6.9% over MMI respectively. Numerical experiments show that increasing number of pickers and picking capacity has a greater impact on reducing makespan when the ratio of orders to number of pickers or picking capacity is high.

1. Introduction

E-commerce sales has seen tremendous growth in recent years. The compound annual growth rate of US e-commerce is estimated to be 9.5% between 2013 and 2018 (Acimovic and Graves, 2014). Considering this significant growth in e-commerce sales, it is imperative to take notice of the major developments in the market. E-commerce operations have extremely challenging profiles; Typical characteristics of e-commerce orders are small orders, each picking, large item count, irregular order arrival patterns, seasonality and high service level expectations (Leung et al., 2018). Therefore, basic warehouse management system (WMS) functionalities are unable to effectively manage such operations. These operations need additional tools and system enhancements to run efficiently. Intelligent systems utilizing operations research and advanced analytics techniques are needed to turbo charge WMS capabilities so that it can be efficient in e-commerce warehouses. There is a need for intelligent systems, which focus on multiple aspects to optimize order fulfillment that minimizes 'Click-to-Ship' time (i.e. order cycle time).

While dealing with a diverse set of activities, including picking, cartonizing, and shipping products to customers (Ardjmand et al., 2018; Kalra et al., 2018), most warehouses utilize a wave-based order picking system where a group of orders (referred to as a wave) placed by customers are simultaneously released to be picked. If the picking time between waves (i.e. picking makespan of waves) is minimized, then the overall picking time can be reduced significantly. Picking time of a wave depends on how the customer orders are batched together (batching problem), how the batches are assigned to the pickers (assignment problem), and how the pickers are routed for order picking (routing problem). To minimize the order picking makespan of a wave, it is necessary to simultaneously solve order batching, assignment and pickers routing. Overall picking process typically represents 60% of total cost in a warehouse operation (Kulak et al., 2012) and approximately 50% of a picker's time is spent over traveling (Tompkins et al., 2010).

To the best of authors' knowledge, there are only two studies, which simultaneously consider order batching, assignment and routing problems. One study conducted by Chen et al. (2015), deals with

* Corresponding author.

E-mail addresses: ardjmand@frostburg.edu (E. Ardjmand), heman@ksu.edu (H. Shakeri), manjeet.singhin@dhl.com (M. Singh), omid.saneibajgiran@rotman.utoronto.ca (O. Sanei Bajgiran).

<https://doi.org/10.1016/j.ijpe.2018.10.001>

Received 26 March 2018; Received in revised form 4 October 2018; Accepted 6 October 2018

Available online 09 October 2018

0925-5273/© 2018 Elsevier B.V. All rights reserved.

minimizing tardiness and as pointed out by Scholz et al. (2017), is limited in scope and cannot be applied to large-scale problems. Another study in this domain is conducted by Scholz et al. (2017) where they minimize tardiness and the method they propose is applicable to large scale problems. However, the problem formulated by Scholz et al. (2017) is not quite well suited for wave picking warehouses where a common objective is to minimize the makespan of picking each wave. Despite the studies by Chen et al. (2015) and Scholz et al. (2017) where sequencing is considered, in this study sequencing is of no direct concern. This is mainly because in wave picking scenarios where minimizing makespan is the objective and the orders do not have separate due dates, sequence of batches picked by a single picker does not alter the overall makespan. However, when each order has a due date, different batch sequences can lead to different tardiness levels.

Following the release of a wave in a warehouse, it is necessary to complete picking before the next wave is released. Thus, warehouse managers try to optimize order assignment, batching and pickers' routing such that it minimizes the makespan of picking a wave. Additionally, minimizing the makespan also has the benefit of balancing the workload among pickers. Order assignment, order batching, pickers' routing and number of available pickers affect the total picking makespan in wave. Most of the studies in the literature approach the order picking problem with the objective of minimizing the total distance traveled by the pickers or order fulfilment tardiness. While minimizing the total distance traveled can reduce the costs of picking, it does not guarantee a balanced workload among pickers and a minimum makespan that is essential in wave picking warehouses. In fact, when minimizing the total distance, having one or multiple pickers is of no concern. This is because when the total distance is minimized, different batches can be assigned to one or multiple pickers while the total distance is at its minimum. This clearly shows that in wave picking scenarios with multiple pickers, total traveled distance is not a suitable objective function.

Similarly, it can be discussed that minimizing order fulfilment tardiness is not usually applicable in wave picking warehouses because it can lead to solutions with unbalanced workloads. To illustrate this point, consider the example of Fig. 1 with two pickers and four batches. Let us assume that all orders (and thus all batches) have the same due date as depicted. If the objective is to minimize the tardiness, then both solutions a and b have 0 tardiness and hence, are equally good. However, solution b has a more balanced workload for the pickers and is generally more acceptable to warehouse managers. When minimizing the makespan, since the maximum picking time of the pickers determine the makespan the solutions will be pushed to have similar completion times for all the pickers. In this regard, minimizing the makespan is usually more appealing to wave picking environments.

In this study, order assignment, order batching and picker routing in a large wave picking warehouse belonging to a major US third party logistic company is investigated. A mathematical model is proposed for the problem. To solve the problem, an exact method using Gurobi 7.5, a Lagrangian decomposition (LD) heuristic that takes advantage of an exact routing method proposed by Ratliff and Rosenthal (1983) and a parallel simulated annealing and ant colony optimization (PSA-ACO) hybrid algorithm are proposed. The results obtained are compared

against the current order picking practices of the aforementioned warehouse and a makespan compatible version of the VND proposed by Scholz et al. (2017).

The contributions of this study are as follows:

- A mathematical model is developed which minimizes order picking makespan in a warehouse of a major US third party logistics company. The warehouse studied employs traditional wave picking with multiple pickers.
- A Lagrangian decomposition heuristic is proposed that performs well for small size waves.
- For large-scale waves, a hybrid parallel simulated annealing and ant colony optimization algorithm is proposed.
- The warehouse data is used to evaluate the solutions from the proposed models and are compared with the output from existing heuristic algorithm used at the warehouse and a makespan compatible version of the VND algorithm proposed by Scholz et al. (2017).

The remainder of this study is arranged as follows. Section 2 is dedicated to the problem description. Section 3 reviews the related literature. In section 4, the mathematical model is proposed. The current practices of the warehouse as well as the methodologies used for solving the proposed model are introduced in section 5. Section 6 shows the numerical experiments and managerial insights and finally section 7 presents the conclusions of the study.

2. Problem description

The warehouse of this study has a rectangular shape. All aisles are parallel and there are two front and rear cross aisles and an I/O point located at the bottom left of the floor (Fig. 2). The objective of this study is to propose effective solutions for minimizing the picking makespan in the aforementioned warehouse with multiple pickers. For this purpose, customers' order data of several working days in the warehouse were used.

The picking operation begins with the pickers starting to pick the batches assigned to them. Each batch contains one or more orders. Fig. 3 depicts a small example of the problem with 10 orders and 2 pickers where each circle represents an order and its items. Each order contains one or more items. Orders are batched together and each batch is assigned to a picker. Makespan is the time when all batches are picked. To pick a batch, a picker must traverse a tour in the warehouse, which visits the location of all items in the batch. As it can be observed in Fig. 3, changing the sequence of batches for a picker does not affect the makespan. However, if the objective was to minimize the tardiness, changing the sequence of batches would alter the value of objective function. There is no limit on the number or duration of batches that are assigned to a picker. Obviously, in an optimum solution, the batches are assigned to the pickers in such a way to minimize the overall makespan. Similar to Scholz et al. (2017), it is assumed that pickers' traveling speed is constant and is not affected by congestion in aisles.

Considering the example of Fig. 3, the problem of current study can be stated as follows: let a non-empty set of customer orders (released in a wave) each containing a set of items with known locations in the

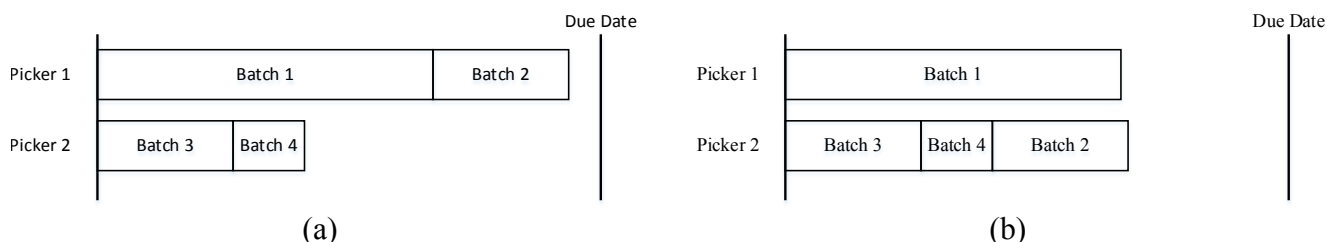


Fig. 1. Minimizing tardiness vs makespan and workload balance.

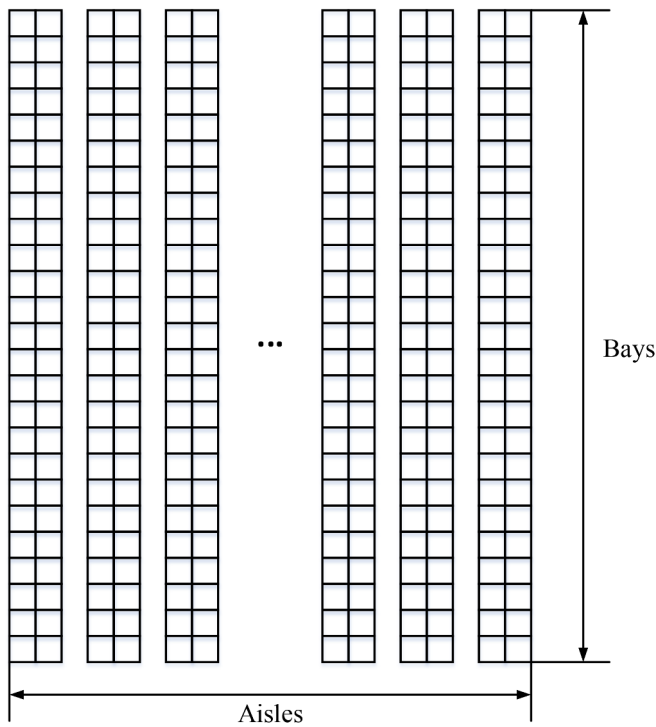


Fig. 2. The studied warehouse.

warehouse and a number of pickers given. Then, the following questions need to be answered in such a way to minimize the makespan:

- How should the set of customer orders be batched together? (order batching problem)
- How should the batches be assigned to the pickers? (assignment problem)
- In which order should the location of items in a batch be visited? (routing problem)

3. Literature review

At an operational level, order picking comprises of order assignment, order batching, sequencing and picker routing. Previous research has addressed order picking problems in various situations (Cergibozan and Tasan, 2016). Table 1 summarizes previous studies, order picking related problems they have considered, warehouse layout studied, methods used, special constraints or situations taken into account and objectives used. It can be easily observed that there are only a few studies that consider makespan as their objective in which only order batching is addressed. Most of studies are focused on total distance/time traveled. The most studied layout is parallel aisle layout with two front and rear cross aisles. Only the study by Scholz et al. (2017) has addressed all order picking problems with the objective of minimizing total tardiness. However, as discussed earlier, the problem formulated by Scholz et al. (2017) is not quite well suited for wave picking warehouses where a common objective is to minimize the makespan of picking each wave. To the best of authors' knowledge there is no study in the literature that simultaneously solves order batching, order assignment and routing while minimizing the picking makespan. In Table 1, the studies that do not address the routing problem directly and use existing routing strategies or previously existing methods are not categorized as routing problem. Moreover, studies that involve tactical level decisions such as item to zone assignment (Kuo et al., 2016) are out of the current's study scope and hence, are not reviewed.

To address the order picking problem with multiple pickers, it is necessary to solve the order assignment problem as well. Table 1 shows that only a few studies have considered the order assignment problem. A common theme among order assignment studies is consideration of tardiness as the objective function. When the objective function is to minimize the total traveled distance, assignment problem is irrelevant, as assigning batches to the pickers does not affect the total traveled distance.

The second sub-problem in order picking operations is order batching. Order batching is the most commonly addressed problem among all the studies in Table 1. Order batching is concerned with grouping customers' orders in such a way to minimize the total traveled distance when picking the items, given a specific routing strategy. For a detailed account of order batching problems and their solution methods

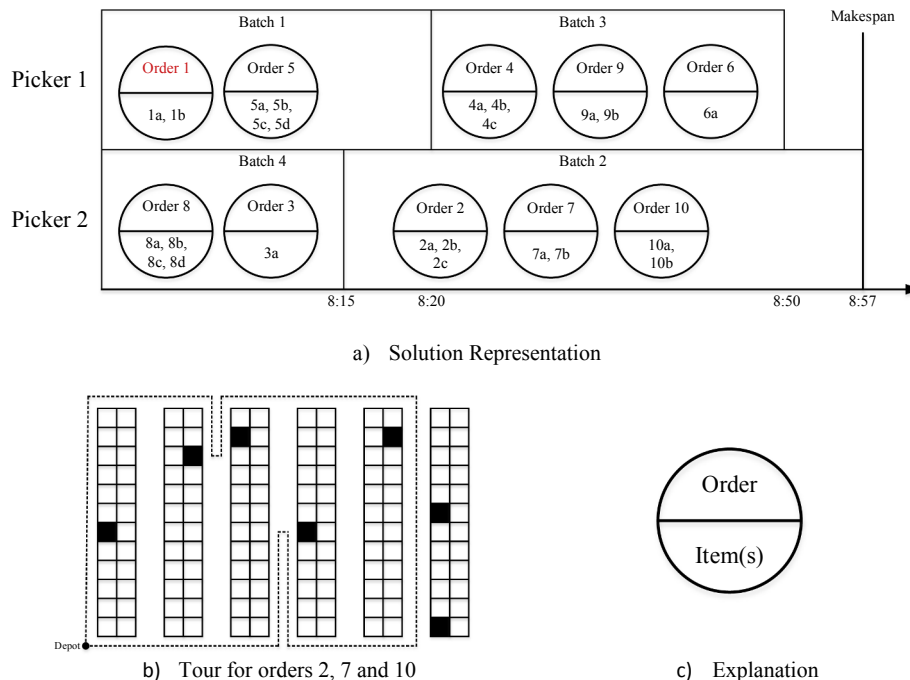


Fig. 3. A small example with 10 orders and 2 pickers.

Table 1
Related studies.

Study	Assignment	Batching	Routing	Sequencing	Layout	Method	Constraint(s)	Objective(s)
Ratliff and Rosenthal, 1983		✓	✓		Parallel aisle	Dynamic programming		Total distance/time
Roodbergen and De Koster, 2001		✓	✓		Parallel aisle and a middle aisle	Dynamic programming		Total distance/time
Gademmann et al., 2001	✓		✓		Parallel aisle	Branch and bound (B&B)		Makespan
Hwang et al., 2004					Parallel aisle	Heuristics		Total distance/time
Chen et al., 2005	✓	✓			Parallel aisle	Association rule mining		Order similarity in Batches
Gademmann and Velde, 2005	✓	✓			Parallel aisle	Branch and price (B&P)		Total distance/time
Hwang and Kim, 2005	✓	✓			Parallel aisle	Cluster analysis		Order similarity in Batches
Won and Olafsson, 2005	✓	✓	✓		Parallel aisle	Heuristic		Response time
Bozer and Kile, 2008	✓	✓			Parallel aisle and one main aisle	Heuristic		Total distance/time
Ho et al., 2008	✓	✓	✓		Parallel aisle	Heuristics		Total distance/time
Tsai et al., 2008	✓	✓	✓		Parallel aisle	Multiple-GA		Total distance/time and due date
Albarca-Sambola et al., 2009	✓	✓			General	Variable neighbor search (VNS)		Total distance/time
Hsieh and Huang, 2011	✓	✓	✓		Parallel aisle with one cross aisle	K-means, self-organizing map (SOM)		Total distance/time and average picking vehicle utility
Henn, 2012	✓	✓			Parallel aisle	Heuristic	Orders received on-line	Makespan
Henn and Wäscher, 2012	✓	✓			Parallel aisle	Attribute based hill climbing (ABHC) and tabu search (TS)		Total distance/time
Hong et al., 2012	✓	✓			Parallel aisle	Heuristic		Total distance/time
Kulak et al., 2012	✓	✓	✓		Single and multiple cross aisles	TS		Total distance/time
Azadnia et al., 2013	✓	✓	✓		Parallel aisle	Association rule mining and genetic algorithm (GA)		Total tardiness
Henn and Schmid, 2013	✓	✓		✓	Parallel aisle	Iterated local search and attribute-based hill climber		Total tardiness
Çelik and Süral, 2014	✓	✓	✓		Fishbone aisle	An exact method		Total distance/time
Matusiak et al., 2014	✓	✓	✓		Parallel aisle with three cross aisles	A*-algorithm and simulated annealing (SA)	Picking precedence	Total distance/time
Chen et al., 2015	✓	✓	✓		Parallel aisle	GA and ant colony optimization (ACO)		Total tardiness
Cheng et al., 2015	✓	✓	✓	✓	Parallel aisle	PSO and ACO		Total distance/time
Henn, 2015	✓	✓	✓	✓	Parallel aisle	Variable neighborhood descent (VND) and VNS		Total tardiness
Öncan, 2015	✓	✓			Parallel aisle	Iterated local search with tabu thresholding		Total distance/time
Pan et al., 2015	✓	✓			Pick-and-pass system	Group GA		Number of batches and workload balance
Chen et al., 2016		✓	✓		Parallel aisle with one cross aisle	ACO	Congestion and nondeterministic picking time	Total distance/time
Isler et al., 2016		✓	✓		Parallel aisle	Heuristic		Total distance/time
Lin et al., 2016	✓	✓	✓		Parallel aisle	PSO	Inventory availability and heterogeneous fleet	Total distance/time
Cortés et al., 2017		✓	✓		Parallel aisle	TS		Total distance/time
Li et al., 2017	✓	✓	✓		Parallel aisle with a middle aisle	Similarity coefficient heuristic and ACO		Total distance/time
Ardjmand and Huh, 2017	✓	✓	✓		Parallel aisle	Non-dominated sorting GA-II (NSGA-II)	Combined with production scheduling	Total distance/time and makespan
Menéndez et al., 2017a	✓	✓		✓	Parallel aisle	VNS		Total tardiness
Menéndez et al., 2017b	✓	✓			Parallel aisle	VNS		Total distance/time
Menéndez et al., 2017c	✓	✓			Parallel aisle	Parallel VNS		Min-max batch retrieval time

(continued on next page)

Table 1 (continued)

Study	Assignment	Batching	Routing	Sequencing	Layout	Method	Constraint(s)	Objective(s)
Pansart et al., 2017			✓		Parallel aisle with multiple middle aisle	Dynamic programming		Total distance/time
Scholz et al., 2017	✓	✓	✓	✓	Parallel aisle with a middle aisle	VNS		Total tardiness
Schrotenboer et al., 2017			✓		Parallel aisle	GA		Total distance/time
Vaile et al., 2017		✓	✓		Parallel aisle with a middle aisle	An exact method	Product returns and interaction delays	Total distance/time
Žulij et al., 2018		✓			General	Adaptive large neighborhood search and TS		Total distance/time

an interested reader is referred to De Koster et al. (2007) and Henn et al. (2012).

The picker routing sub-problem is concerned with arranging the sequence of items that are to be picked in a tour in order to minimize the total length of the tour. Picker routing is a special case of traveling salesman problem (TSP). It is shown that in warehouses with up to two blocks, picker routing can be efficiently solved to optimality (Ratliff and Rosenthal, 1983; Roodbergen and De Koster, 2001). However, when combined with other order picking sub-problems such as order batching where it is necessary to solve the routing problem repeatedly, exact methods are not usually fast enough (Scholz et al., 2017). For this reason, many heuristic approaches are proposed for the picker routing problem. Traversal, midpoint and the largest gap heuristics are among some of the well-known picker routing heuristics (De Koster et al., 2007). For a detailed account of order batching problems and their solution methods an interested reader is referred to De Koster et al. (2007).

The batch sequencing problem deals with sequencing batches assigned to a picker in order to minimize the total tardiness of orders. As the definition of problem suggests, sequencing is applicable to tardiness minimization problems. As discussed by Scholz et al. (2017), a special case of the batch sequencing problem where each batch contains one single order is extensively studied in the parallel machine sequencing literature.

Study of the order picking literature shows that in recent years researchers have become more interested in addressing more than one order picking sub-problem simultaneously. Chen et al. (2015) considered the batching, routing and sequencing problems for minimizing the total tardiness. They used a hybrid genetic algorithm (GA) to solve the batching and sequencing problems and ACO to solve the routing problem. The order picking problem formulated by Chen et al. (2015) is quite limited for practical purposes as it considers only one picker (thus no assignment problem is needed) and is able to solve small problems of up to only 8 orders.

As discussed earlier, the study by Scholz et al. (2017) is the only study that simultaneously considers all four order picking sub-problems. The authors use a VND algorithm for solving the assignment, batching and sequencing problems and take advantage of two combined (Roodbergen and De Koster, 2001) and Lin-Kernighan-Helsgaun (LKH) heuristics (Helsgaun, 2000) for the routing problem. They show that their method is quite effective in solving larger instances of the problem (up to 200 orders). Since the VND algorithm of Scholz et al. (2017) is not compatible to the problems where the objective is makespan minimization, a modified version of it is used in this study for comparison purposes, which will be discussed in more details in section 5.

Although the order picking sub-problems are addressed in different settings, the review of the literature shows that there is no study regarding makespan minimization while considering all relevant sub-problems. The objective of this study is to fill this gap while providing an efficient solution that can be used for real-life applications.

4. Model

In this section, mathematical model of the problem is proposed. Consider a rectangular parallel aisle warehouse where N items are stored. Each item occupies one location (i.e. bay) and each location is dedicated to only one item. There are R pickers that start picking the orders as soon as a wave is released. There are O orders in the wave and each picker can have maximum B batches each including maximum C_{unit} units of products or C_{order} orders. The objective is to batch orders, assign them to pickers and route pickers in order to minimize makespan of picking all batches. Indices, parameters, decision variables and the model are as follows:

Indices:

$b \in \{1, 2, \dots, B\}$: Batch index
 $o \in \{1, 2, \dots, O\}$: Order index
 $i, j, k \in \{1, 2, \dots, N\} \cup \{G\}$: Items index (including the origin G)
 $r \in \{1, 2, \dots, R\}$: Picker index

Parameters:

C_{unit} : maximum number of units allowed to be assigned to a picker in a batch

C_{order} : maximum number of orders allowed to be assigned to a picker in a batch

d_{ij} : minimum traveling time between locations of items i and j

q_{oi} : quantity of item i in order o

M : Sufficiently large number

Decision Variables:

x_{bo}^r : 1 if order o is assigned to batch b of picker r

y_{bij}^r : 1 if item i is picked right before item j in batch b of picker r

t_{bi}^r : time of picking item i by picker r in batch b

T_b^r : completion time of batch T traversed by picker r

MS_r : makespan of each picker

C_{max} : total makespan

Model M :

$$\text{Min } C_{max} \quad (1)$$

$$\sum_{b=1}^B \sum_{r=1}^R x_{bo}^r = 1 \quad \forall o \in \{1, 2, \dots, O\} \quad (2)$$

$$\sum_{o=1}^O x_{bo}^r \leq C_{order} \quad \forall b \in \{1, 2, \dots, B\}, r \in \{1, 2, \dots, R\} \quad (3)$$

$$\sum_{o=1}^O \sum_{i=1}^N x_{bo}^r q_{oi} \leq C_{unit} \quad \forall b \in \{1, 2, \dots, B\}, r \in \{1, 2, \dots, R\} \quad (4)$$

$$x_{bo}^r q_{oj} \leq M \sum_{i=1, i \neq j}^N y_{bij}^r \quad \forall b \in \{1, 2, \dots, B\}, o \in \{1, 2, \dots, O\}, j \in \{1, 2, \dots, N\}, r \in \{1, 2, \dots, R\} \quad (5)$$

$$\sum_{i=1}^N y_{bij}^r = \sum_{k=1}^N y_{bjk}^r \quad \forall b \in \{1, 2, \dots, B\}, j \in \{1, 2, \dots, N\}, r \in \{1, 2, \dots, R\} \quad (6)$$

$$\sum_{i=1}^N y_{bij}^r \leq 1 \quad \forall b \in \{1, 2, \dots, B\}, j \in \{1, 2, \dots, N\}, r \in \{1, 2, \dots, R\} \quad (7)$$

$$t_{bj}^r \geq d_{oj} - M(1 - y_{boj}^r) \quad \forall b \in \{1, 2, \dots, B\}, j \in \{1, 2, \dots, N\}, r \in \{1, 2, \dots, R\} \quad (8a)$$

$$t_{bj}^r \geq t_{bi}^r + d_{ij} - M(1 - y_{bij}^r) \quad \forall b \in \{1, 2, \dots, B\}, i \in \{1, 2, \dots, N\}, j \in \{1, 2, \dots, N\}, r \in \{1, 2, \dots, R\} \quad (8b)$$

$$T_b^r \geq t_{bi}^r + d_{iG} y_{biG}^r \quad \forall b \in \{1, 2, \dots, B\}, i \in \{1, 2, \dots, N\}, r \in \{1, 2, \dots, R\} \quad (9)$$

$$MS_r \geq \sum_{b=1}^B T_b^r \quad \forall r \in \{1, 2, \dots, R\} \quad (10)$$

$$C_{max} \geq MS_r \quad \forall r \in \{1, 2, \dots, R\} \quad (11)$$

Objective function of model M minimizes the total makespan. Constraint (2) guarantees that in a feasible solution all orders are picked. If this constraint is not included, all x_{bo}^r s can be set to 0 and hence, the makespan will be 0. Constraints (3) and (4) are capacity constraints and ensure that a batch does not contain more than a certain number of units or orders. Constraint (5) guarantees that if an order is included in a picker's batch, the location of that item will be a part of the picker's route. In other words, this constraint guarantees that all

items will be eventually included in pickers' tours.

Constraint (6) ensures that the number of times a picker meets a location in a tour is equal to the number of times the picker leaves the location. This constraint is necessary to preserve the consistency of tours created and generate feasible practical tours. Without constraint (6), the solutions obtained may include unfinished or partial tours. Constraint (7) limits the number of times a location is met during a tour to one. Constraints (8) prevents sub-tours. Constraints (6), (7) and (8) together guarantee a consistent set of complete tours that do not violate capacity feasibility. These types of constraints are quite common in the order picking literature (Chen et al., 2015; Scholz et al., 2017). Constraint (9) measures the termination time of each tour. Constraint (10) finds the makespan for each picker and constraint (11) determines the total makespan. Constraints (9), (10) and (11) together link the length of tours to the makespan of each individual picker and then the overall makespan.

5. Methodologies

In this section, the current methodology used in the warehouse as well as the proposed LD-PSO and PSA-ACO algorithms are explained. Additionally, the VND algorithm proposed by Scholz et al. (2017) is briefly introduced and its application to the current problem is explained.

5.1. Current practice of the warehouse under study

In its current situation, the warehouse under study utilizes a heuristic (referred to as minimum makespan impact (MMI)) to solve the order assignment, batching and routing problems. To solve, first a picker-order matrix is generated that stores the makespan changes that occur due to assignment of an order to a picker. Initially, no order is assigned to the pickers and each picker has one empty batch. Using the picker-order matrix, at each iteration the order that will minimize the impact on the makespan of picking assigned orders will be chosen and assigned to the picker with sufficient capacity. If no such picker exists, a new empty batch for all pickers will be created and then the orders will be assigned according to the picker-order matrix. To evaluate the impact of assigning an order to a picker, it is necessary to compute the tour lengths for all pickers. This is done using a traversal routing strategy (Hwang et al., 2004). Fig. 4 shows a hypothetical picker-matrix for assigning 5 orders to 4 pickers where orders o_2 and o_4 are already assigned and cannot impact the makespan anymore (shown as NA). In the matrix, the lowest makespan impact occurs if order 3 is assigned to the picker 1. If picker 1 does not have sufficient capacity, then the next lowest impact assignment (i.e. order 5 to picker 3) will be chosen. Every time an order assignment occurs, the picker-order matrix has to be updated. If no picker has the capacity to incorporate a new order, a new empty batch will be created for each picker and the assignment process will follow. Algorithm 1 shows the pseudo code for the current picking heuristic of the warehouse under study. Note that wherever necessary, a traversal routing strategy is used to compute the makespan.

	o_1	o_2	o_3	o_4	o_5
r_1	7	NA	5	NA	12
r_2	12	NA	4	NA	11
r_3	8	NA	15	NA	6
r_4	13	NA	12	NA	9

Fig. 4. A hypothetical picker-order matrix in which a value in cell (r, o) represents the makespan increase if order o is assigned to picker r .

Algorithm 1

current picking heuristic of the warehouse under study.

```

create an empty batch for each picker
form a picker-order matrix
WHILE all orders are not assigned:
    found = FALSE
    WHILE there are unexplored unassigned orders AND not found:
        choose an unassigned order and a picker with minimum
        makespan impact
        IF the chosen picker has sufficient capacity in his current
        batch:
            assign the order to the picker
            found = TRUE
            update picker-order matrix using a traversal routing
            strategy
        IF found = FALSE:
            create a new batch for each picker

```

5.2. Lagrangian Decomposition and particle swarm optimization

Lagrangian relaxation (LR) is a well-known decomposition algorithm that strives to compute dual bounds of the optimal solution by benefiting from the inherent structure of the problems (Geoffrion, 1974). This algorithm was originally introduced by Everett (Everett, 1963) and became popular with its successful application to large-scale traveling salesman problems (Held and Karp, 1971). In LR, complicated constraints are relaxed and their possible violations are penalized in the objective function of the relaxed model (i.e., the LR model). In other words, the violation of relaxed constraints is incorporated into the objective function by introducing dual multipliers. This relaxation of complicated constraints helps to solve the original problem faster. After considering dual multipliers in the original model, the relaxed model transforms to a nonlinear programming model. There are different approaches to solve this nonlinear model among which the sub-gradient method is most widely used. The optimization of Lagrangian multipliers can be interpreted as optimizing the primal objective function on the intersection of the convex hulls of constraint sets.

Lagrangian Decomposition (LD) is a special case of LR in which an optimization problem is decomposed into two or more disjoint problems (Guignard and Kim, 1987). In other words, if there are two or more sub-problems linked with several common constraints, one can relax these constraints and decompose the problem into several independent sub-problems. LD differs from other relaxation algorithms because every constraint in the original problem appears in one of the sub-problems. The bound predicted by LD is at least as tight as the one provided by LR (Guignard and Kim, 1987).

In model M (referred to as the master problem), it is possible to relax constraint (5) and decompose the problem into two sub problems SP1 and SP2 as follows:

Sub problem 1 (SP1):

$$\text{Min } Z_{SP1} = \sum_{b=1}^B \sum_{o=1}^O \sum_{j=1, j \neq G}^N \sum_{r=1}^R \theta_{bojr} x_{bo}^r q_{oj} \quad (12)$$

$$\sum_{b=1}^B \sum_{r=1}^R x_{bo}^r = 1 \quad \forall o \in \{1, 2, \dots, O\} \quad (13)$$

$$\sum_{o=1}^O x_{bo}^r \leq C_{order} \quad \forall b \in \{1, 2, \dots, B\}, r \in \{1, 2, \dots, R\} \quad (14)$$

$$\sum_{o=1}^O \sum_{i=1}^N x_{bo}^r q_{oi} \leq C \quad \forall b \in \{1, 2, \dots, B\}, r \in \{1, 2, \dots, R\} \quad (15)$$

Sub problem 2 (SP2):

$$\text{Min } Z_{SP2} = C_{max} - M \sum_{b=1}^B \sum_{o=1}^O \sum_{j=1, j \neq G}^N \sum_{r=1}^R \theta_{bojr} \sum_{i=1, i \neq j}^N y_{bij}^r \quad (16)$$

$$\sum_{i=1, i \neq j}^N y_{bij}^r = z_{bj}^r \quad \forall b \in \{1, 2, \dots, B\}, j \in \{1, 2, \dots, N\}, r \in \{1, 2, \dots, R\} \quad (17)$$

$$\sum_{i=1}^N y_{bij}^r = \sum_{k=1}^N y_{bjk}^r \quad \forall b \in \{1, 2, \dots, B\}, j \in \{1, 2, \dots, N\}, r \in \{1, 2, \dots, R\} \quad (18)$$

$$\sum_{i=1}^N y_{bij}^r \leq 1 \quad \forall b \in \{1, 2, \dots, B\}, j \in \{1, 2, \dots, N\}, r \in \{1, 2, \dots, R\} \quad (19)$$

$$t_{bj}^r \geq d_{Gj} - M(1 - y_{bGj}^r) \quad \forall b \in \{1, 2, \dots, B\}, j \in \{1, 2, \dots, N\}, r \in \{1, 2, \dots, R\} \quad (20)$$

$$t_{bj}^r \geq t_{bi}^r + d_{ij} - M(1 - y_{bij}^r) \quad \forall b \in \{1, 2, \dots, B\}, i \in \{1, 2, \dots, N\}, j \in \{1, 2, \dots, N\}, r \in \{1, 2, \dots, R\} \quad (21)$$

$$T_b^r \geq t_{bi}^r + d_{iG} y_{biG}^r \quad \forall b \in \{1, 2, \dots, B\}, i \in \{1, 2, \dots, N\}, r \in \{1, 2, \dots, R\} \quad (22)$$

$$MS_r \geq \sum_{b=1}^B T_b^r \quad \forall r \in \{1, 2, \dots, R\} \quad (23)$$

$$C_{max} \geq MS_r \quad \forall r \in \{1, 2, \dots, R\} \quad (24)$$

where $\theta_{bojr} \in \Theta$ is the Lagrangian multiplier connecting SP1 and SP2. SP1 solves order assignment and batching problems while SP2 solves the routing problem. Note that after assignment and batching are determined in SP1, in order to solve SP2, the routing problem is solved for each picker separately. SP1 and SP2 can be solved separately. Thus, instead of solving model $M1$, it is possible to solve SP1 and SP2 sequentially and separately. However, this requires Lagrangian multipliers to be updated every time SP1 and SP2 are solved. SP1 can be solved using a commercial software (e.g. Gurobi). Given the layout of the warehouse under study, SP2 can be solved for each picker and in polynomial time using the dynamic programming approach presented by Ratliff and Rosenthal (1983). Lagrangian decomposition leads to creation of a Lagrangian dual problem (LDP) which can be defined as follows:

$$Z_{LDP} = \max_{\theta \geq 0} (Z_{SP1}(\Theta) + Z_{SP2}(\Theta)) \quad (25)$$

Where Z_{LDP} is the optimum value of master problem's objective function. To find optimum Z_{LDP} , it is sufficient to find the proper Θ .

As already mentioned, there are several iterative methods such as sub-gradient algorithm to solve the proposed LDP. In this study, initially a sub-gradient algorithm was applied to find Θ iteratively. However, soon it was found that the algorithm suffers from very low convergence rate that makes it unsuitable for practical purposes. Thus, a standard PSO algorithm was utilized to update Θ . Each particle in the PSO is a vector whose components are θ_{bojr} . The objective function of each particle is calculated by sequentially solving SP1 and SP2. The

steps involved in the proposed LD-PSO are shown in Algorithm 2.

Algorithm 2
LD-PSO algorithm.

Relax constraint $x_{bo}^r q_{oj} \leq M \sum_{i=1, i \neq j}^N y_{bij}^r$ in model M Define a dual multiplier θ_{bojr}
 Add the multiplication of dual multipliers and relaxed constraints into the objective function
 Decompose the original master problem into two sub-problems (SP1 and SP2)
 Establish the Lagrangian dual problem (LDP)
 Generate an initial population of particles $p \in \{1, 2, \dots, P\}$ **WHILE** termination criteria is not met:
 FOR each $p \in \{1, 2, \dots, P\}$:
 Calculate the fitness value (makespan) by solving SP1 and SP2
 Update each particle's position and velocity using the following equations:

$$V_{pd}(\tau + 1) = w(\tau)V_{pd}(\tau) + U(0, \phi_1)(pbest_{pd}(\tau) - X_{pd}(\tau))$$

$$+ U(0, \phi_2)(gbest_d(\tau) - X_{pd}(\tau))$$

In Algorithm 2, τ is iteration index, p is particle index, d is dimension index, $U(0, \phi_1)$ and $U(0, \phi_2)$ are uniformly distributed random numbers, $V_{pd}(\tau)$ is the velocity of particle p at the p th dimension in iteration τ , $w(\tau)$ is the inertia weight in τ iteration, $X_{pd}(\tau)$ is the position of the p th particle at the d th dimension in the τ th iteration, $pbest_{pd}(\tau)$ is the personal best position of the p th particle at the d th dimension in the τ th iteration and $gbest_d(\tau)$ is the global best position of the d th dimension. Termination criteria in this study is based on the number of iterations taken or a maximum CPU time reached.

5.3. Parallel hybrid simulated annealing and ant colony optimization

To solve the order assignment, batching and picker routing problems, hybrid simulated annealing (SA) and ant colony optimization (ACO) algorithms are used. SA is a metaheuristic, which works based on statistical mechanics principles. Starting from a random solution x , in each step of the SA a new neighbor solution y is explored. If y is better in terms of solution quality, it will be accepted as the new solution, otherwise it will be accepted based on the acceptance probability $p = e^{\frac{-(f(y)-f(x))}{t}}$ in which t is the temperature parameter. As the SA algorithm moves forward, t will be reduced gradually and it becomes harder to accept poor quality solutions. The process of decreasing t is called cooling and can be done in various ways. Two commonly used methods in the literature are to decrease t by a fixed number or a specified percentage (geometric cooling schedule) until a low temperature is achieved. SA is very sensitive to cooling schedule and a generic cooling schema might not work for all the problems. Usually an adaptive cooling schedule is used (Ingber, 2000). In this study, an adaptive SA, namely list-based simulated annealing (LBSA) is used for solving order batching and assignment. This algorithm is based on the ideas proposed by Zhan et al. (2016).

ACO mimics collective behavior of ants in nature where the objective is to find the shortest path from their nest to the food source (Dorigo and Birattari, 2011). In this study, ACO implementation closely follows the one utilized by Cheng et al. (2015) and is used to solve the routing problem which is modeled as a Steiner TSP (Fig. 5). In the Steiner TSP of Fig. 5 black vertices are the pick locations and the depot and have to be included in the tour while white vertices that represent the intersection of aisles do not necessarily need to be visited in a tour. Maximum degree of a node in the layout of Fig. 5 is 3. Thus, the Steiner TSP of this layout has fewer edges than a standard TSP. An optimum

route of this Steiner TSP is a tour that starting from the depot, visits all black vertices along the depicted edges and has the minimum possible length.

A solution to the order assignment, batching and picker routing problems can be represented by collection of R vectors where R is the number of pickers. Each vector includes the orders assigned to corresponding picker. Starting from the first element in each vector, orders are batched together if capacity constraints are not violated. If an order cannot be added to the current batch due to capacity constraints, a new batch is created for the picker and the order is added to the new batch. This process continues until all orders of a picker are batched. After all batches are determined, a routing solution for each batch is obtained using ACO. Fig. 6 illustrates a solution representation with 15 orders and 3 pickers. Each picker's vector contains the orders that a picker needs to pick. Considering the batch capacity (in terms of number of orders or units), orders are batched together from left to right. After batching, tour length for each batch is optimized and calculated using ACO. In Fig. 6, C_{max} represent the time after a new wave is released (wave release) (containing 15 orders) until all batches are picked. PSA-ACO has three stages as follows.

5.3.1. Initialization

Initial order assignment is based on the solution obtained using current heuristic employed at the warehouse. At this stage, the solution contains no batching information and simply assigns each order to a picker. It is possible to initialize the assignment of orders randomly as well. However, numerical experiments showed that good initial starting points, such as the one explained, can improve the quality of final solutions.

5.3.2. Batching and routing

After initialization, orders are batched together using LBSA. Batching is performed for each picker independently and in parallel. This allows for substantial CPU time savings. As mentioned earlier, each picker has an assignment vector that is batched from left to right while considering capacity constraints. Thus, the same set of orders when arranged differently, can lead to alternate batching solutions. To illustrate this, consider picker r_1 in Fig. 6 where his assignment vector is (2,4,7,12,9,14,1). Orders assigned to r_1 are batched together from left to right considering batch capacity of maximum 3 orders. Thus, batches created are {(2, 4, 7), (12, 9, 14), (1)}. Now, if the assignment vector was arranged as (4,12,9,14,2,7,1), while it contains the same orders, it would result in a different batching solution, i.e. {(4, 12, 9), (14, 2, 7), (1)}. To evaluate different batching solutions, the optimum route for each batch is obtained using ACO and the solution with minimum overall length is selected.

LBSA is used to find the best batching solution for each picker. For this purpose, starting from an initial assignment vector ω , a neighbor assignment vector ω' is generated using inverse, insert and swap operations (Wang et al., 2015; Zhan et al., 2016). Inverse operation works by inverting the sequence of orders between positions i and j in an assignment vector ω . If the i th position of ω is shown by $\omega(i)$, then the inverse operation using positions i and j creates a new assignment vector ω' such that $\omega'(i) = \omega(j)$, $\omega'(i+1) = \omega(j-1)$, ..., $\omega'(j) = \omega(i)$. Note that $1 \leq i, j \leq n$ and $1 \leq j-i < n-1$ where n is the assignment vector length. Insert operation moves an order in position j of an assignment vector to position i ($j > i$). Thus, by applying the insert operation on ω , a new assignment vector will be generated such that $\omega'(i) = \omega(j)$, $\omega'(i+1) = \omega(i)$, ..., $\omega'(j) = \omega(j-1)$. Swap operation swaps two orders i and j . Thus, by applying the swap operation on assignment vector ω a new assignment vector will be generated such that $\omega'(i) = \omega(j)$ and $\omega'(j) = \omega(i)$.

In LBSA the first step is to produce an initial temperature list L . Temperature list includes temperature levels that should be used in the cooling schedule. For a given temperature level, it is possible to calculate the probability of accepting a worse neighbor solution. Similarly,

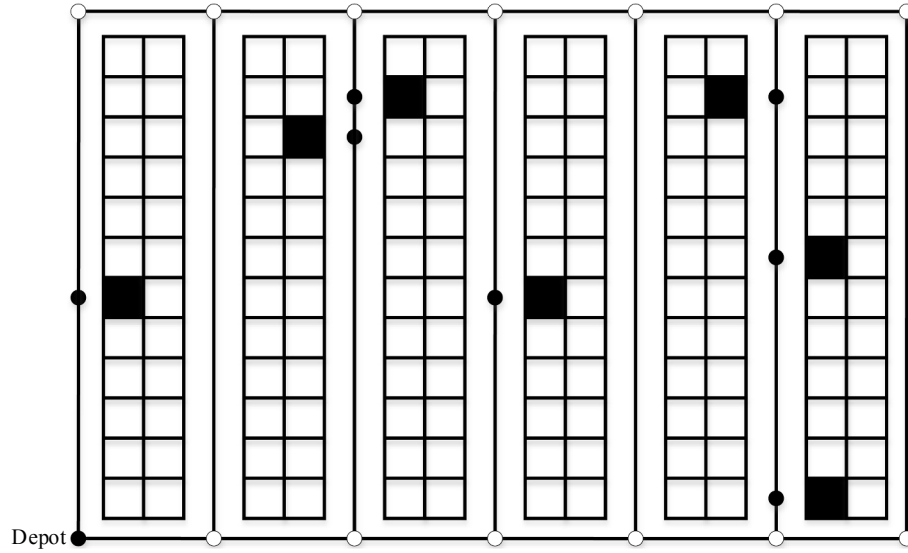


Fig. 5. A Steiner TSP.

by inverting the acceptance probability formula and having the initial acceptance probability p_0 , it is possible to calculate the corresponding temperature as $t = \frac{-(f(y)-f(x))}{\ln p_0}$. The temperature levels obtained using p_0 will be used to create an initial temperature list. Algorithm 3 shows the pseudo-code for producing an initial temperature list.

Algorithm 3
pseudo-code for producing an initial temperature list.

```

Create an initial solution  $x$ 
Define temperature list  $L = []$  # initialize an empty list
Define the length of temperature list  $L_{max}$ 
Define initial acceptance probability  $p_0$ 
Create an initial solution  $x$ 
while length ( $L$ )  $\leq L_{max}$ 
    create a neighbor solution  $y$  from  $x$ 
    if  $f(y) \leq f(x)$ : ...#  $f$  is the objective function
         $x = y$ 
    else:
         $t = \frac{-(f(y)-f(x))}{\ln p_0}$ 

```

In each iteration of the LBSA, the maximum temperature t_{max} in the L will be used for calculating the probability of accepting a worse solution. In a LBSA, I is number of times a neighbor solution is generated in an iteration and $MS(\omega)$ is the total makespan of assignment vector ω which is calculated using ACO. Algorithm 4 depicts the pseudo-code of the LBSA. The temperature list L updates in each iteration of LBSA.

Algorithm 4
LBSA pseudo-code.

```

Define  $g_{best}$  as the best found solution with  $MS(g_{best}) = +\infty$ 
Generate an initial order assignment vector  $\omega$ 
Produce an initial temperature list  $L$ 
Define number of iterations  $n$ 
Define number of neighbor solutions generated in each iteration  $I$ 
for  $i$  in (1 to  $n$ ):
     $c = 0$  # number of worse solutions accepted
     $t_{sum} = 0$  # summation of temperature levels
    for  $m$  in (1 to  $I$ ):
        generate a neighbor assignment vector  $\omega'$ 
        if  $MS(\omega') \leq MS(\omega)$ :
             $\omega = \omega'$ 
            if  $MS(\omega) \leq MS(g_{best})$ :
                 $g_{best} = \omega$ 
        else:
             $p = e^{\frac{-(MS(\omega')-MS(\omega))}{t}}$  #  $p$  is the acceptance probability
            Generate a random number  $\alpha \in [0,1)$ 
            if  $\alpha < p$ :
                 $c = c + 1$ 
                 $t_{sum} = t_{sum} + \frac{-(MS(\omega')-MS(\omega))}{\ln \alpha}$ 
                 $x = y$ 
            if  $c \neq 0$ :

```

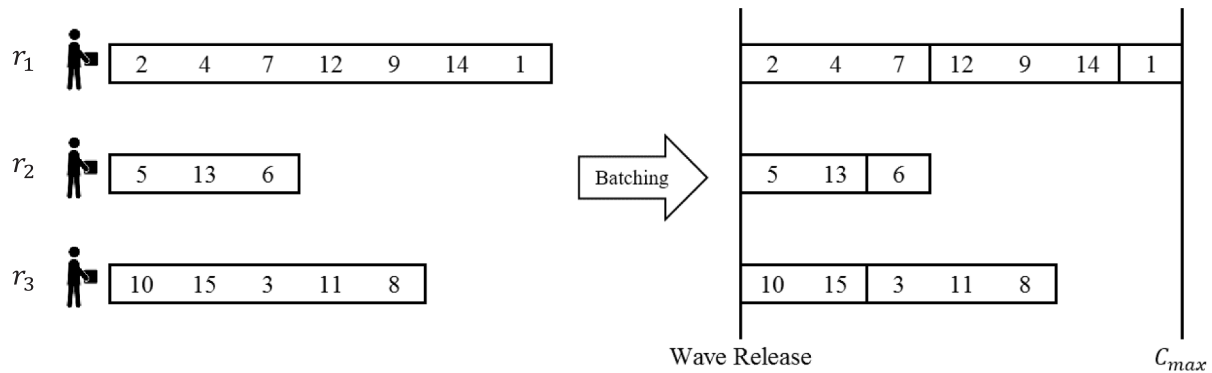


Fig. 6. Solution representation.

5.3.3. Reassigning batches

After orders are assigned and batched, PSA-ACO reassigns the batches with objective of reducing total makespan. This is achieved using another LBSA that is structurally similar to the one explained previously. To reassign batches, two batches in the current batch assignment solution are chosen and swapped. In other words, a neighborhood solution is generated through swapping batches between pickers. Beside the neighborhood structure, all other elements of LBSA at this stage are similar to the one explained earlier. Note that swapping of batches between pickers will not modify the corresponding batch lengths. However, it changes the related pickers' and total makespan. Thus, after reassigning batches, it is not necessary to invoke ACO for routing and recalculating the makespan. Fig. 7 depicts how a new batch assignment solution is generated by swapping the second batch of picker r_1 with the second batch of picker r_2 in the example shown in Fig. 6.

After reassigning batches, each picker will have a new order assignment vector which can be further improved by going back to the stage two of PSA-ACO. This process can continue until no further improvement is realized. Fig. 8 shows the flowchart of PSA-ACO.

5.4. Variable neighborhood descent

One of the most successful algorithms for minimizing tardiness of order picking with multiple pickers is the VND proposed by Scholz et al. (2017) where they simultaneously solve order batching, batch assignment and sequencing, and picker routing. Since the VND is proposed for minimizing tardiness, it is not directly applicable to the problem of this study where the objective is to minimize makespan. However, with some modifications, the VND can be used for makespan minimization. In this section, only the modifications made to the VND are mentioned. An interested reader is referred to (Scholz et al., 2017) for more information on overall structure of the VND.

The first modification made to the VND is initial solution construction. Originally, VND utilizes an earliest start date rule to construct an initial solution. While this rule is quite successful for tardiness minimization, it does not yield promising solutions for makespan minimization. Thus, similar to PSA-ACO, initial solution construction of VND is obtained using the current heuristic employed at the warehouse.

The second modification to the VND is for calculating optimum pickers' routes. Original VND, utilizes two methods of combined heuristic and Lin-Kernighan-Helsgaun (LKH) heuristic. Note that combined and LKH heuristics are not intrinsic components of a VND algorithm. Thus, they can be replaced by any other routing algorithm while keeping the overall structure of VND intact. To have a fair and more meaningful comparison between PSA-ACO and VND, the routing procedure of VND is replaced by the one used by PSA-ACO. The third modification made to the VND is concerned with how a solution is represented and the objective function is calculated. All solution representations and objective function calculations of the VND are modified to match the makespan minimization problem studied here.

6. Numerical experiments

In this section, data from a major US third party logistics company is used to evaluate the performance of exact method, LD-PSO and PSA-ACO and compare it against the current heuristic used. For this purpose, five days are picked randomly and their transactional level data is gathered. The warehouse of this study has 23 parallel aisles and 25 bays in each aisle side. Fig. 9 depicts the order profile of the warehouse in terms of lines per order, number of times an item is ordered, units per line (a distinct article) and units per order. As it can be observed, majority of orders have less than 5 lines and 25 units. Also, majority of lines contain less than 15 units and there only a few items that are ordered more than 40 times during the investigated 5 days. Every day one wave, is released early morning 6 a.m., which includes all the

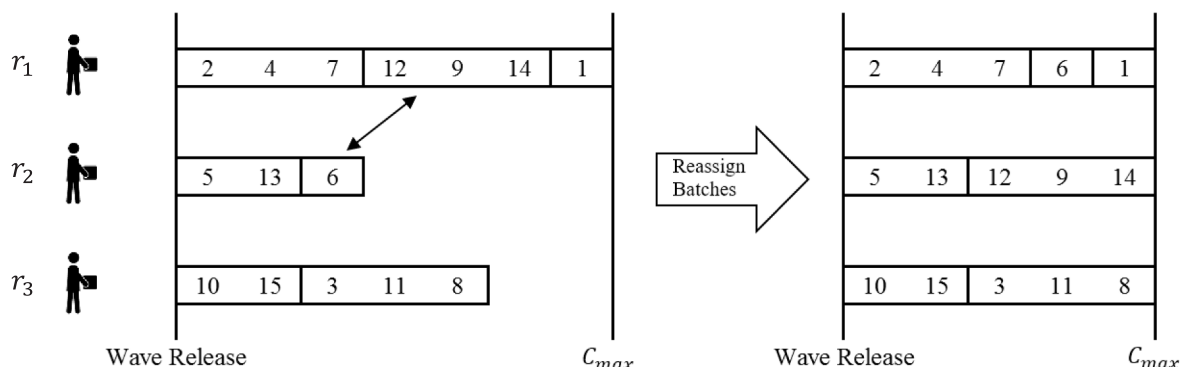


Fig. 7. Reassigning batches.

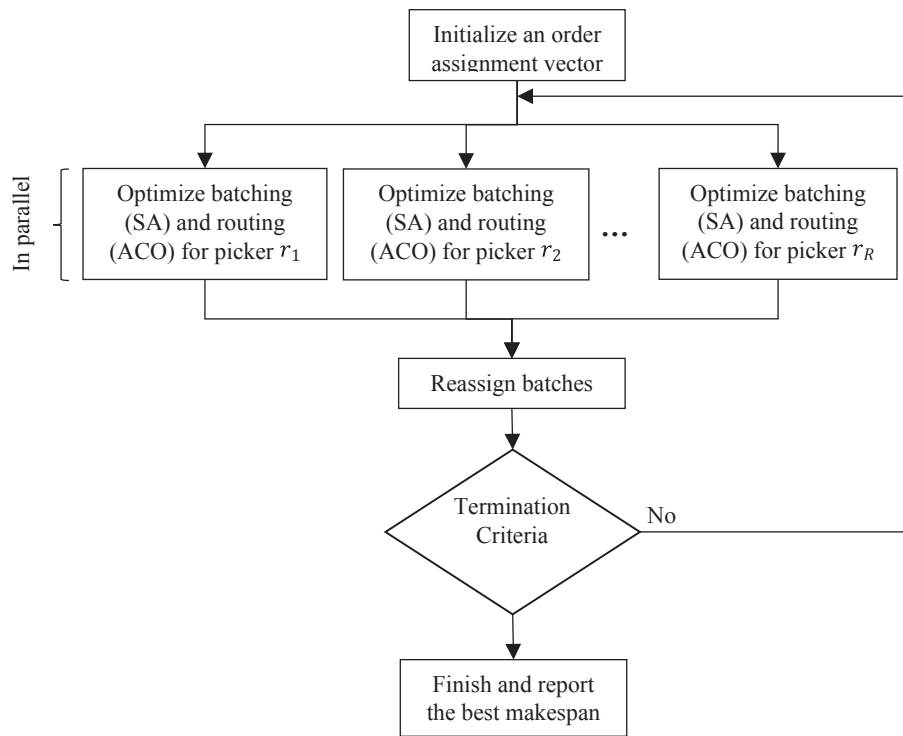


Fig. 8. PSA-ACO flowchart.

orders that have to be picked that day. It is usually preferred to pick the orders as soon as possible so that after picking, pickers can be assigned to other warehouse operations. Table 2 shows number of orders received each day.

Before being released, a wave is constantly changing. If a wave is to be released on 6:00 a.m., new orders are being received from the customers until minutes before the release time and thus, the wave is being constantly modified before 6:00 a.m. In this situation, one has to wait until the wave has its final form and then solve the order picking sub problems (i.e. assignment, batching, routing and sequencing problems). Thus, in practice there is a limited time to address these problems. Usually in warehouses there is a cut off time for each wave. For

instance, if the cut off is 10 min, all orders that are received up to 10 min before the wave release time will be included in the wave. In this scenario, there is only 10 min available for solving the order picking sub problems. In the case of this paper, this cut off is approximately 30 min. Thus, a 30 min (1800 s) CPU time limit is chosen.

The numerical experiments of this section are composed of two parts. First, in order to evaluate the gaps of MMI, LD-PSO, PSA-ACO and VND to the exact solutions, 5 small datasets, including 5, 10, 15, 20 and 25 orders are generated by sampling the existing orders. Small sampled instances approximately have an average of 3 items per order and 6 units per order. Next, to evaluate the VND and the PSA-ACO for large instances, they are applied to the large original dataset. Due to CPU time limitations and NP-hardness of the problem under investigation, LD-PSO and the exact method cannot be applied to the original large datasets. CPU time for all instances is limited to 1800 s. Each small and large test problems are solved 10 times and the best, average and the worst solutions found are reported. The computer used for the experiments had a 3.50 GHz Xeon CPU and 64.0 GB RAM. Note that all of the times reported in this study are in seconds, except if stated otherwise. The makespan in this study is measured in seconds.

6.1. Small instances

Number of pickers in small instances is chosen from {2,3,4}. Capacity of pickers is two orders or five units. Table 3 shows the results of experimentations on small instances obtained by sampling from the actual data. As it can be observed, only in three instances Gurobi has

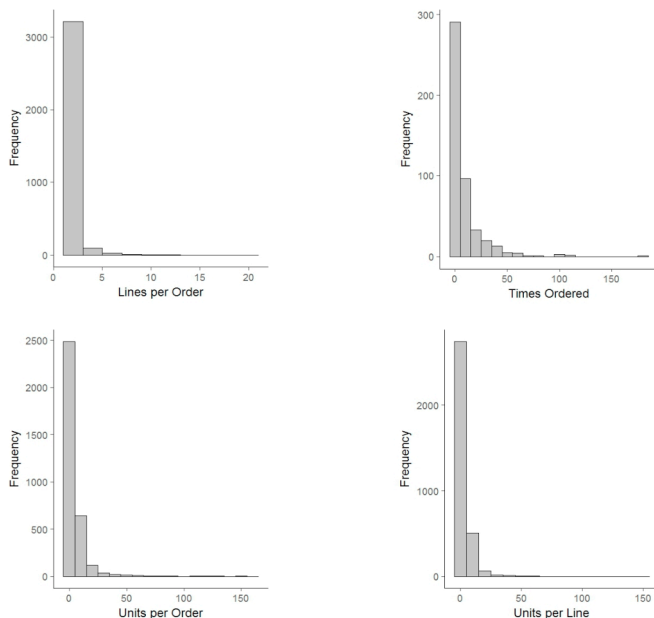


Fig. 9. Order profile of the warehouse.

Table 2
Number of orders received in each day.

Day	Number of Orders
1	776
2	1093
3	865
4	1001
5	924

Table 3
Results of experimentations on small instances.

Orders	Pickers	MMI		Exact		LD-PSO			PSA-ACO				VND				
		Makespan	CPU (t)	Makespan	CPU (t)	Min	Avg.	Max	CPU (t)	Min	Avg.	Max	CPU (t)	Min	Avg.	Max	CPU (t)
5	2	1014	< 1	962	644	962	962	962	33	962	962	962	38	962	962	962	44
	3	954	< 1	854	705	854	854	854	34	854	854	854	38	854	854	854	44
	4	854	< 1	720	699	720	720	720	29	720	720	720	32	720	720	720	38
10	2	1858	< 1	1146	1800	1146	1146	1146	97	1146	1146	1146	40	1146	1146	1146	40
	3	1192	< 1	892	1800	892	892	892	102	892	892	892	37	892	892	892	49
	4	1162	< 1	768	1800	768	768	768	101	768	768	768	42	768	768	768	43
15	2	2016	< 1	1960	1800	1856	1856	1856	328	1856	1856	1856	40	1856	1856	1856	58
	3	1566	< 1	1314	1800	1292	1292	1292	331	1292	1292	1292	37	1292	1292	1292	53
	4	1194	< 1	992	1800	992	992	992	386	992	992	992	41	992	992	992	52
20	2	3694	< 1	4430	1800	3600	3627	3638	1071	3558	3564	3578	69	3517	3548	3569	61
	3	3046	< 1	NA	1800	2712	2819	2972	1058	2524	2531	2545	61	2531	2535	2539	64
	4	2424	< 1	NA	1800	2154	2232	2316	1196	1946	1950	1950	62	1927	1933	1946	67
25	2	3604	< 1	NA	1800	3598	3702	3822	1800	3150	3154	3158	57	3194	3209	3212	82
	3	2930	< 1	NA	1800	2778	2951	3032	1800	2038	2045	2048	52	2042	2057	2065	75
	4	2062	< 1	NA	1800	2006	2189	2370	1800	1532	1543	1550	53	1529	1540	1548	84

Table 4
Results of experimentations on large instances.

Orders	Pickers	MMI	PSA-ACO				VND				
		Makespan	CPU (t)	Min	Avg.	Max	CPU (t)	Min	Avg.	Max	CPU (t)
776	4	11,748	9	10,746	10,825	10,930	796	10,850	10,915	10,960	1231
	6	9228	9	7918	8080	8166	716	8044	8121	8199	1363
	8	7412	8	6224	6367	6446	792	6386	6410	6455	1223
1093	4	14,682	13	13,872	13,903	14,004	837	14,023	14,088	14,174	1570
	6	11,384	14	10,214	10,366	10,462	832	10,177	10,317	10,475	1543
	8	9154	15	8416	8524	8606	796	8365	8493	8602	1688
865	4	11,864	9	11,504	11,568	11,678	803	11,776	11,797	11,813	1370
	6	8896	11	8378	8518	8650	777	8458	8665	8767	1371
	8	7286	11	6658	6702	6764	765	6603	6669	6806	1399
1001	4	14,366	12	13,060	13,150	13,196	828	13,146	13,381	13,571	1471
	6	10,762	13	9580	9674	9838	784	9728	9897	9973	1458
	8	8454	14	7632	7760	7814	768	7600	7755	7900	1480
924	4	12,892	11	12,078	12,139	12,256	757	12,392	12,458	12,508	1479
	6	9606	11	8822	8954	9008	745	8909	8977	9092	1422
	8	7890	11	7224	7334	7494	749	7297	7422	7582	1387

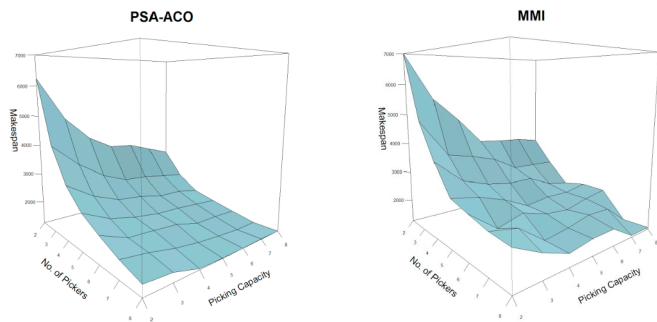


Fig. 10. Makespan changes vs no. of pickers and picking capacity for PSA-ACO and MMI.

successfully found the optimum solution in 1800 s where LD-PSO and PSA-ACO have also found the optimal. LD-PSO has been able to find an optimum or good solution compared to the exact solution in a much shorter CPU time, which shows efficiency of the proposed Lagrangian decomposition heuristic. However, PSA-ACO and VND have successfully found optimum or the best solutions in a much shorter time. One notable observation regarding the PSA-ACO is its slow CPU time increment as the problem size increases. This property is a result of SA parallelization, which makes PSA-ACO a suitable candidate for large-scale real life problems. Although other solutions performed better than MMI, its CPU time was below 1 s for all small instances tried. MMI

outperforms average results of LD-PSO for test problems with 25 orders. However, the best solutions found by LD-PSO for the 25 orders instances is superior to the MMI solutions. On average, LD-PSO, PSA-ACO and VND outperform MMI by approximately 11.9%, 18.6% and 18.5% respectively.

No significant difference between the solutions' quality of PSA-ACO and VND was found for small instances of the problem. However, VND CPU time was found to be significantly longer than PSA-ACO. This difference is not due to VND inefficiency. A part of PSA-ACO is implemented in parallel, which creates an edge over VND. In general, VND neighborhood structures cannot be implemented in parallel because they modify multiple pickers in each step. One reason PSA-ACO can be paralleled is that when working on sequencing and routing problems, PSA-ACO optimizes the sequence of each picker separately. This property can create significant benefits for large instances of the problem.

6.2. Large instances

Number of pickers for large instances is chosen from {4,6,8} which is according to the number of warehouse pickers in different days. Capacity values are set on $C_{order} = 12$ and $C_{unit} = 168$ according to the warehouse manager's suggestion. Table 4 shows the results of experimentations on large instances. The PSA-ACO and VND are better than MMI by approximately 7.8% and 6.9% respectively. 7.8% improvement over the MMI can lead to an estimated \$125,000 annual savings in the studied warehouse. Additionally, CPU time of the proposed PSA-ACO is

Table 5
Difference between the smallest and largest workload among pickers for large problems.

Orders	Pickers	PSA-ACO	VND
776	4	138	196
	6	131	174
	8	163	193
1093	4	142	230
	6	204	174
	8	179	171
865	4	133	151
	6	130	193
	8	168	163
1001	4	140	168
	6	177	203
	8	173	164
924	4	176	194
	6	138	163
	8	175	186

reasonable for such large-scale problems. The relatively short CPU time is mainly due to the parallelization of SA in the proposed PSA-ACO, which contributed to approximately 70% savings in the computation time.

Another notable advantage of PSA-ACO is the relatively small difference between the best and worst solutions it found. This is an important property especially for large real life problems as it makes the solutions obtained more reliable. Interestingly, it was observed that MMI solutions are not always reliable as they may lead to unintentional results. For instance, it is generally expected to get a lower makespan as the number of pickers or pickers' capacity increases. We observed that on some occasions by increasing the number of pickers and/or capacity it yielded relatively higher makespan.

It was found that on average, PSA-ACO yields slightly better results compared to VND (approximately 1%). However, VND CPU time is significantly higher than PSA-ACO. This is mainly due to parallelization properties of PSA-ACO and the fact that most of neighborhood structures in VND cannot be implemented in parallel. While for cases with 4 and 6 pickers PSA-ACO and VND have the largest performance difference, For cases with 8 pickers, there is no evidence of significant performance difference between PSA-ACO and VND.

Fig. 10 depicts makespan changes vs number of pickers and picking capacity for PSA-ACO and MMI in a randomly created test problem by sampling 50 orders from the actual orders. It can be observed in Fig. 10 that PSA-ACO has a predictable behavior as the number of pickers and their capacity increases. In the case of MMI, while the general trend of decreasing makespan for increased number of pickers and capacity holds true, in some cases a decrease in any of pickers number or capacity can slightly worsen the makespan. This example clearly shows the reliability of PSA-ACO.

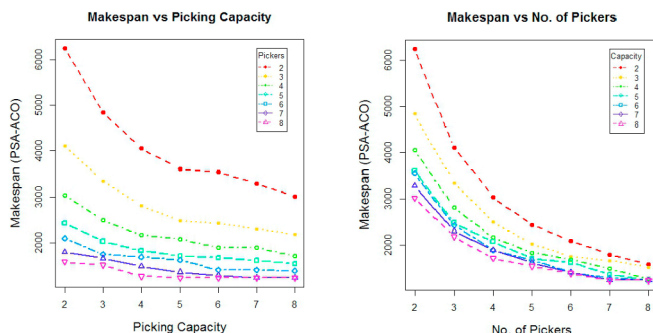


Fig. 11. Makespan changes for different values of pickers number and capacity.

6.3. Workload balance analysis

As briefly pointed out earlier, minimizing makespan has a secondary benefit of balancing workload among pickers. Intuitively speaking, when a decision maker minimizes the makespan, reducing overall travel distance/time of a single picker does not improve the objective function. To improve the makespan, it is necessary to reduce the travel distance/time for all pickers, which balances the workload among pickers. When minimizing tardiness or total travel distance/time, this property is not necessarily observed. Table 5 lists the difference between the smallest and largest workload (i.e. traveled distance) among pickers for large problems. On average, PSA-ACO workload difference is 1.6% of the total makespan while this number for VND is 1.8%. This shows that the difference between the pickers' workloads is quite small and minimizing the makespan can balance the workload as well.

6.4. Determining the number of pickers and pickers' capacity in a warehouse

Besides increasing the picking operation efficiency, the proposed model and methods as well as obtained results can be utilized to gain some practical insights. In this section, some observations and practical suggestions for reducing the makespan of picking a wave are presented.

After developing the model and methods an interesting problem that authors faced was how to determine the number of pickers and their picking capacity using this model. While determining the number of pickers and their capacity depends on many factors such as cost of hiring a new employee and investing on picking vehicles, it should be noted that under different situations the same amount of investment can yield different makespan savings. This is because the relationship between makespan and number of pickers and capacity is not linear. This notion was tested using a randomly created test problem by sampling 50 orders from real warehouse orders. Fig. 11 shows change in makespan as number of pickers and capacity increases. Adding more pickers and capacity can lead to larger makespan savings when the problem is highly constrained, i.e. the ratio of orders to pickers/capacity is high. This can be observed in Fig. 11 where in some cases adding to the picking capacity has approximately no effect on the makespan. This specifically is evident in cases when there are 4 or more pickers. This effect can also be explained considering the role of makespan minimization in balancing the workload of pickers as was discussed previously.

By increasing number of pickers by one, a fraction of pickers' workload will be transferred to the newly added picker. Thus, if already there are many pickers in the warehouse, each has to contribute only a small part of his workload to the newly added picker in order to balance the workload and minimize the makespan. Conversely, if there are a few pickers in the warehouse, each has to contribute a larger share of his workload to the newly added picker in order to balance the workload. Thus, when there are few pickers available, adding a new picker leads to larger makespan reduction. Similarly, when picking capacity is high, adding to this capacity does not affect the makespan as much as when the picking capacity is low.

7. Conclusion

In this study, order assignment, order batching and picker routing in a large wave picking warehouse from a major US third party logistics company was studied. A mathematical model for the problem was developed and two algorithms (LD-PSO and PSA-ACO) were proposed for solving the model. The methods proposed were tested on the data obtained from the warehouse and compared against MMI algorithm being utilized at the warehouse and a state-of-the-art VND. It was observed that the suggested LD-PSO is much more effective in terms of computing time than the exact method. However, it is not suitable for large-scale real instances due to infeasible CPU time. Additionally, it was found that PSA-ACO outperforms all other algorithms in terms of

solution quality and is quite reliable for large-scale problems observed in the warehouse. Evaluation of the proposed PSA-ACO showed that it can lead to an average 7.8% makespan improvement over the MMI. This could potentially lead to approximately \$125,000 annual savings. While testing the methods on warehouse data, it was observed that increasing number of pickers and capacity does not decrease the makespan linearly. Increasing number of pickers and capacity will generate more makespan-savings if the ratio of orders to number of pickers and capacity is high.

This study points to various problems worth investigating in the future. In the studied warehouse, after the items are picked, they are usually processed (i.e. packaged, assembled, etc.) further before shipping. Considering this extra layer of activities while optimizing the makespan can improve the overall picking and processing operations. Additionally, the presented model can be integrated with other operational level decisions that proceed it such as distribution and transporting to customers. Along similar lines, the presented model can be integrated with strategic decision making (e.g. warehouse layout). This is especially important when one considers the impact of layout on length of tours traversed by each picker. Finally, considering the proposed model in a multiple warehouse scenario, where the orders first have to be assigned to one of the warehouses in a distribution network, seems to be a logical extension to this problem. In a multiple warehouse setting, there are more than one warehouse for fulfilling the customers' orders. In this case, first, orders are assigned to warehouses based on their inventory and proximity to the customers' locations and then the orders are released in each warehouse as a wave. In a multiple warehouse setting, beside order picking sub-problems, the problem of assigning orders to warehouses should be addressed as well.

References

- Acimovic, J., Graves, S.C., 2014. Making better fulfillment decisions on the fly in an online retail environment. *Manuf. Serv. Oper. Manag.* 17, 34–51.
- Albareda-Sambola, M., Alonso-Ayuso, A., Molina, E., De Blas, C.S., 2009. Variable neighborhood search for order batching in a warehouse. *Asia Pac. J. Oper. Res.* 26, 655–683.
- Ardjmand, E., Bajgiran, O.S., Rahman, S., Weckman, G.R., Young, W.A., 2018. A multi-objective model for order cartization and fulfillment center assignment in the e-tail/retail industry. *Transport. Res. Part E Logist. Transport. Rev.* 115, 16–34.
- Ardjmand, E., Huh, D.W., 2017. Coordinated warehouse order picking and production scheduling: a NSGA-II approach. In: *Computational Intelligence (SSCI), 2017 IEEE Symposium Series on*. IEEE, pp. 1–8.
- Azadnia, A.H., Taheri, S., Ghadimi, P., Mat Saman, M.Z., Wong, K.Y., 2013. Order batching in warehouses by minimizing total tardiness: a hybrid approach of weighted association rule mining and genetic algorithms. *Sci. World J.* 2013.
- Bozer, Y.A., Kile, J.W., 2008. Order batching in walk-and-pick order picking systems. *Int. J. Prod. Res.* 46, 1887–1909.
- Çelk, M., Süral, H., 2014. Order picking under random and turnover-based storage policies in fishbone aisle warehouses. *IIE Trans.* 46, 283–300.
- Cergibozan, Ç., Tasan, A.S., 2016. Order batching operations: an overview of classification, solution techniques, and future research. *J. Intell. Manuf.* 1–15.
- Chen, F., Wang, H., Xie, Y., Qi, C., 2016. An ACO-based online routing method for multiple order pickers with congestion consideration in warehouse. *J. Intell. Manuf.* 27, 389–408.
- Chen, M.-C., Huang, C.-L., Chen, K.-Y., Wu, H.-P., 2005. Aggregation of orders in distribution centers using data mining. *Expert Syst. Appl.* 28, 453–460.
- Chen, T.-L., Cheng, C.-Y., Chen, Y.-Y., Chan, L.-K., 2015. An efficient hybrid algorithm for integrated order batching, sequencing and routing problem. *Int. J. Prod. Econ.* 159, 158–167.
- Cheng, C.-Y., Chen, Y.-Y., Chen, T.-L., Yoo, J.J.-W., 2015. Using a hybrid approach based on the particle swarm optimization and ant colony optimization to solve a joint order batching and picker routing problem. *Int. J. Prod. Econ.* 170, 805–814.
- Cortés, P., Gómez-Montoya, R.A., Muñizuri, J., Correa-Espinal, A., 2017. A tabu search approach to solving the picking routing problem for large-and medium-size distribution centres considering the availability of inventory and K heterogeneous material handling equipment. *Appl. Soft Comput.* 53, 61–73.
- De Koster, R., Le-Duc, T., Roodbergen, K.J., 2007. Design and control of warehouse order picking: a literature review. *Eur. J. Oper. Res.* 182, 481–501.
- Dorigo, M., Birattari, M., 2011. *Ant Colony Optimization*, Encyclopedia of Machine Learning. Springer, pp. 36–39.
- Everett III, H., 1963. Generalized Lagrange multiplier method for solving problems of optimum allocation of resources. *Oper. Res.* 11, 399–417.
- Gademann, A., Van Den Berg, J.P., Van Der Hoff, H.H., 2001. An order batching algorithm for wave picking in a parallel-aisle warehouse. *IIE Trans.* 33, 385–398.
- Gademann, N., Velde, S., 2005. Order batching to minimize total travel time in a parallel-aisle warehouse. *IIE Trans.* 37, 63–75.
- Geoffrion, A.M., 1974. *Lagrangean Relaxation for Integer Programming*, Approaches to Integer Programming. Springer, pp. 82–114.
- Guignard, M., Kim, S., 1987. Lagrangean decomposition: a model yielding stronger Lagrangean bounds. *Math. Program.* 39, 215–228.
- Held, M., Karp, R.M., 1971. The traveling-salesman problem and minimum spanning trees: Part II. *Math. Program.* 1, 6–25.
- Helsgaun, K., 2000. An effective implementation of the Lin–Kernighan traveling salesman heuristic. *Eur. J. Oper. Res.* 126, 106–130.
- Henn, S., 2012. Algorithms for on-line order batching in an order picking warehouse. *Comput. Oper. Res.* 39, 2549–2563.
- Henn, S., 2015. Order batching and sequencing for the minimization of the total tardiness in picker-to-part warehouses. *Flex. Serv. Manuf. J.* 27, 86–114.
- Henn, S., Koch, S., Wäscher, G., 2012. Order Batching in Order Picking Warehouses: a Survey of Solution Approaches, Warehousing in the Global Supply Chain. Springer, pp. 105–137.
- Henn, S., Schmid, V., 2013. Metaheuristics for order batching and sequencing in manual order picking systems. *Comput. Ind. Eng.* 66, 338–351.
- Henn, S., Wäscher, G., 2012. Tabu search heuristics for the order batching problem in manual order picking systems. *Eur. J. Oper. Res.* 222, 484–494.
- Ho, Y.-C., Su, T.-S., Shi, Z.-B., 2008. Order-batching methods for an order-picking warehouse with two cross aisles. *Comput. Ind. Eng.* 55, 321–347.
- Hong, S., Johnson, A.L., Peters, B.A., 2012. Large-scale order batching in parallel-aisle picking systems. *IIE Trans.* 44, 88–106.
- Hsieh, L.-F., Huang, Y.-C., 2011. New batch construction heuristics to optimise the performance of order picking systems. *Int. J. Prod. Econ.* 131, 618–630.
- Hwang, H., Kim, D., 2005. Order-batching heuristics based on cluster analysis in a low-level picker-to-part warehousing system. *Int. J. Prod. Res.* 43, 3657–3670.
- Hwang, H., Oh, Y., Lee, Y., 2004. An evaluation of routing policies for order-picking operations in low-level picker-to-part system. *Int. J. Prod. Res.* 42, 3873–3889.
- Ingber, L., 2000. Adaptive Simulated Annealing (ASA): Lessons Learned. arXiv preprint cs/0001018.
- Isler, C., Righetto, G., Morabito, R., 2016. Optimizing the order picking of a scholar and office supplies warehouse. *Int. J. Adv. Manuf. Technol.* 87, 2327–2336.
- Kalra, A., Rangwala, M., Ardjmand, E., DeJesse, C., 2018. Decision support system for optimizing the unit identifier stocking decision. Google Patents.
- Kulak, O., Sahin, Y., Taner, M.E., 2012. Joint order batching and picker routing in single and multiple-cross-aisle warehouses using cluster-based tabu search algorithms. *Flex. Serv. Manuf. J.* 24, 52–80.
- Kuo, R., Kuo, P., Chen, Y.R., Zulueta, F.E., 2016. Application of metaheuristics-based clustering algorithm to item assignment in a synchronized zone order picking system. *Appl. Soft Comput.* 46, 143–150.
- Leung, K., Choy, K., Siu, P.K., Ho, G., Lam, H., Lee, C.K., 2018. A B2C e-commerce intelligent system for re-engineering the e-order fulfilment process. *Expert Syst. Appl.* 91, 386–401.
- Li, J., Huang, R., Dai, J.B., 2017. Joint optimisation of order batching and picker routing in the online retailer's warehouse in China. *Int. J. Prod. Res.* 55, 447–461.
- Lin, C.-C., Kang, J.-R., Hou, C.-C., Cheng, C.-Y., 2016. Joint order batching and picker Manhattan routing problem. *Comput. Ind. Eng.* 95, 164–174.
- Matusiak, M., de Koster, R., Kroon, L., Saarinen, J., 2014. A fast simulated annealing method for batching precedence-constrained customer orders in a warehouse. *Eur. J. Oper. Res.* 236, 968–977.
- Menéndez, B., Bustillo, M., Pardo, E.G., Duarte, A., 2017a. General variable neighborhood search for the order batching and sequencing problem. *Eur. J. Oper. Res.* 263, 82–93.
- Menéndez, B., Pardo, E.G., Alonso-Ayuso, A., Molina, E., Duarte, A., 2017b. Variable neighborhood search strategies for the order batching problem. *Comput. Oper. Res.* 78, 500–512.
- Menéndez, B., Pardo, E.G., Sánchez-Oro, J., Duarte, A., 2017c. Parallel variable neighborhood search for the min-max order batching problem. *Int. Trans. Oper. Res.* 24, 635–662.
- Öncan, T., 2015. Milp formulations and an iterated local search algorithm with tabu thresholding for the order batching problem. *Eur. J. Oper. Res.* 243, 142–155.
- Pan, J.C.-H., Shih, P.-H., Wu, M.-H., 2015. Order batching in a pick-and-pass warehousing system with group genetic algorithm. *Omega* 57, 238–248.
- Pansart, L., Catusse, N., Cambazard, H., 2017. Exact Algorithms for the Picking Problem. arXiv preprint arXiv:1703.00699.
- Ratcliff, H.D., Rosenthal, A.S., 1983. Order-picking in a rectangular warehouse: a solvable case of the traveling salesman problem. *Oper. Res.* 31, 507–521.
- Roodbergen, K.J., De Koster, R., 2001. Routing order pickers in a warehouse with a middle aisle. *Eur. J. Oper. Res.* 133, 32–43.
- Scholz, A., Schubert, D., Wäscher, G., 2017. Order picking with multiple pickers and due dates-simultaneous solution of order batching, batch Assignment and sequencing, and picker routing problems. *Eur. J. Oper. Res.* 263, 461–478.
- Schrotenboer, A.H., Wruck, S., Roodbergen, K.J., Veenstra, M., Dijkstra, A.S., 2017. Order picker routing with product returns and interaction delays. *Int. J. Prod. Res.* 55, 6394–6406.
- Tompkins, J.A., White, J.A., Bozer, Y.A., Tanchoco, J.M.A., 2010. *Facilities Planning*. John Wiley & Sons.

- Tsai, C.-Y., Liou, J.J., Huang, T.-M., 2008. Using a multiple-GA method to solve the batch picking problem: considering travel distance and order due time. *Int. J. Prod. Res.* 46, 6533–6555.
- Valle, C.A., Beasley, J.E., da Cunha, A.S., 2017. Optimally solving the joint order batching and picker routing problem. *Eur. J. Oper. Res.* 262, 817–834.
- Wang, C., Lin, M., Zhong, Y., Zhang, H., 2015. Solving travelling salesman problem using multiagent simulated annealing algorithm with instance-based sampling. *Int. J. Comput. Sci. Math.* 6, 336–353.
- Won, J., Olafsson, S., 2005. Joint order batching and order picking in warehouse operations. *Int. J. Prod. Res.* 43, 1427–1442.
- Zhan, S.-h., Lin, J., Zhang, Z.-j., Zhong, Y.-w., 2016. List-based simulated annealing algorithm for traveling salesman problem. *Comput. Intell. Neurosci.* 8 2016.
- Žulj, I., Kramer, S., Schneider, M., 2018. A hybrid of adaptive large neighborhood search and tabu search for the order-batching problem. *Eur. J. Oper. Res.* 264, 653–664.